

Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №4
по дисциплине «Математическая логика и теория
формальных языков»
«Доработка компилятора языка Milan»
Вариант №30

Выполнила студентка группы
№5130201/30101
Проверил

Лаишевкина А. М. _____
Востров А. В. _____

«____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Описание языка Milan	4
1.2 Анализ генерации случайных чисел в языках программирования	5
1.3 Грамматика языка Milan	6
1.3.1 Контекстно-свободные грамматики	6
1.3.2 Грамматика языка Milan	7
1.3.3 Синтаксическая диаграмма	7
2 Особенности реализации	10
2.1 Определение новых токенов	10
2.2 Добавление инструкций в генератор кода	12
2.3 Расширение синтаксического анализатора	13
2.4 Реализация инструкций в виртуальной машине	15
2.5 Доработка файлов виртуальной машины	20
3 Результаты работы программы	22
Заключение	25
Список литературы	26

Введение

Целью данной работы является доработка компилятора языка программирования Milan путем добавления генерации случайного числа равномерного и нормального распределений.

Задачи:

1. Проанализировать реализацию генерации случайных чисел равномерного и нормального распределений в нескольких языках программирования.
2. Построить БНФ и синтаксическую диаграмму языка Милан с учетом добавленных конструкций.
3. Доработать компилятор и виртуальную машину для обработки добавленных конструкций.

1 Математическое описание

1.1 Описание языка Milan

Язык Milan – учебный язык программирования. Программа на нем представляет собой последовательность операторов, заключенных между ключевыми словами **Begin** и **End**. Операторы отделяются друг от друга точкой с запятой. После последнего оператора в блоке точка с запятой не ставится. Компилятор SMilan не учитывает регистр символов в именах переменных и ключевых словах.

В базовую версию языка Milan входят следующие конструкции: константы, идентификаторы, арифметические операции над целыми числами, операторы чтения чисел со стандартного ввода и вывода чисел на стандартный вывод, оператор присваивания, условный оператор, оператор цикла с условием.

Компилятор SMilan включает три компонента:

1. лексический анализатор;
2. синтаксический анализатор;
3. генератор команд виртуальной машины Милана.

Лексический анализатор посимвольно читает из входного потока текст программы и преобразует группы символов в лексемы (терминальные символы грамматики языка Милан). При формировании лексем анализатор идентифицирует ключевые слова и последовательности символов (например, оператор присваивания), а также определяет значения числовых констант и имена переменных.

Синтаксический анализатор читает сформированную лексическим анализатором последовательность лексем и проверяет ее соответствие грамматике языка Милан. В процессе анализа программы синтаксический анализатор генерирует набор машинных команд, соответствующие каждой конструкции языка.

Генератор кода представляет собой служебный компонент, ответственный за формирование внешнего представления генерируемого кода.

Компилятор SMilan работает по однократному принципу, иными словами синтаксический анализ и генерация кода выполняется в рамках одного цикла обработки.

Виртуальная машина Milan включает в себя области памяти для хранения команд и данных, а также стек для промежуточных вычислений. Виртуальная машина обеспечивает выполнение арифметических операций, условных и безусловных переходов, процедур ввода-вывода, а также работу с целочисленными значениями. Процесс выполнения программы продолжается непрерывно до появления команды завершения или до обнаружения ошибки.

1.2 Анализ генерации случайных чисел в языках программирования

Для обоснования выбранного способа реализации генерации псевдослучайных чисел в языке Milan ниже рассматриваются три языка: C/C++, Pascal и Python.

C / C++

Стандартная библиотека C предоставляет функцию `rand()`, возвращающую целое равномерно распределённое псевдослучайное число из диапазона $[0, \text{RAND_MAX}]$. Для получения числа из произвольного диапазона $[a, b]$ программист вручную вычисляет `a + rand() % (b - a + 1)`.

Нормальное распределение в стандартной библиотеке C не предусмотрено; оно реализуется вручную, как правило методом Бокса–Мюллера:

$$z_0 = \sqrt{-2 \ln u_1} \cos(2\pi u_2), \quad u_1, u_2 \sim \mathcal{U}(0, 1),$$

где u_1 и u_2 — два независимых равномерных числа. Результат сдвигается и масштабируется: $x = \mu + z_0 \cdot \sigma$.

Инициализация генератора выполняется отдельным вызовом `srand(seed)`; при `seed = 0` принято использовать `srand(time(NULL))`, чтобы получить различные последовательности при каждом запуске.

Таким образом, в C каждая из трёх операций (равномерное число, нормальное число, установка зерна) представляет собой отдельный самостоятельный вызов.

Pascal

Стандартная библиотека Pascal предоставляет функцию `Random(N)`, возвращающую случайное целое из $[0, N)$, и `Random` без аргумента — вещественное из $[0, 1)$. Инициализация генератора выполняется либо процедурой `Randomize` (от системного времени), либо прямым присваиванием `RandSeed := value`.

Нормальное распределение в стандартных модулях отсутствует и реализуется обычно тем же методом Бокса–Мюллера.

Pascal, как и C, разделяет получение случайного числа и установку зерна на два независимых синтаксических элемента.

Python

Модуль `random` стандартной библиотеки Python предоставляет три отдельные функции: `random.randint(a, b)` — равномерное целое из $[a, b]$, `random.normalvariate(mu, sigma)` — нормально распределённое вещественное, `random.seed(x)` — установка зерна.

При `x = None` генератор инициализируется от системного времени.

Конструкции, добавленные в Милан

Во трёх рассмотренных языках прослеживаются схожие черты реализации генерации псевдослучайных чисел:

1. Равномерное и нормальное распределения являются разными операциями с совершенно разными параметрами.
2. Установка зерна генератора семантически отличается от получения числа (она влияет на весь последующий поток случайных значений) и выносится в отдельную операцию.

На основании проведённого анализа в язык Милан были добавлены три конструкции:

- `rand(a, b)` – возвращает равномерно распределённое целое из $[a, b]$; оба параметра передаются как произвольные арифметические выражения и вычисляются в стеке;
- `norm(mu, sigma)` – возвращает целое, полученное из нормального распределения $\mathcal{N}(\mu, \sigma^2)$ методом Бокса–Мюллера и округлённое до ближайшего целого;
- `seed(value)` – устанавливает зерно генератора; при значении 0 используется системное время (`srand(time(NULL))`), что повторяет поведение Pascal-процедуры `Randomize` и Python-вызова `random.seed(None)`.

1.3 Грамматика языка Milan

1.3.1 Контекстно-свободные грамматики

Контекстно-свободные грамматики (КС-грамматики) относятся ко второму типу грамматик по классификации Хомского. Эти грамматики описываются productions вида $A \rightarrow \beta$, где A – нетерминал, а β – последовательность терминалов и нетерминалов.

LL(k)-грамматики позволяют выполнять нисходящий синтаксический анализ, просматривая входную цепочку слева направо для построения канонического левого вывода. Параметр k указывает, сколько символов из непрочитанной части входной цепочки используется для принятия решений. LL(1)-грамматики, например, используют только один символ для этого.

Грамматика классифицируется как грамматика рекурсивного спуска, когда для каждого нетерминального символа создается синтаксическая диаграмма, в которой выбор альтернативы в точках ветвления осуществляется на основе одного текущего терминального символа.

Разбор левого нетерминала реализуется посредством иерархической системы вызовов процедур: каждому нетерминальному символу соответствует отдельная процедура, которая включает вызовы других процедур и проверки терминальных символов в последовательности, определенной правилами грамматики. Стек вызовов процедур аналогичен стеку магазинного автомата, используемого в LL(1)-парсере.

1.3.2 Грамматика языка Milan

Ниже приведена грамматика расширенной версии языка Milan в форме Бэкуса–Наура. Внесенные изменения выделены цветом.

```
 $\langle program \rangle ::= \text{begin } \langle statementList \rangle \text{ end}$   
 $\langle statementList \rangle ::= \langle statement \rangle ; \langle statementList \rangle \mid \varepsilon$   
 $\langle statement \rangle ::= \langle identifier \rangle := \langle expression \rangle$   
                   $\mid \text{if } \langle relation \rangle \text{ then } \langle statementList \rangle [\text{else } \langle statementList \rangle] \text{ fi}$   
                   $\mid \text{while } \langle relation \rangle \text{ do } \langle statementList \rangle \text{ od}$   
                   $\mid \text{write } ( \langle expression \rangle )$   
                   $\mid \text{seed } ( \langle expression \rangle )$   
 $\langle expression \rangle ::= \langle term \rangle \{ \langle addop \rangle \langle term \rangle \}$   
                   $\langle term \rangle ::= \langle factor \rangle \{ \langle mulop \rangle \langle factor \rangle \}$   
 $\langle factor \rangle ::= \langle number \rangle \mid \langle identifier \rangle \mid ( \langle expression \rangle ) \mid - \langle factor \rangle \mid \text{read}$   
                   $\mid \text{rand } ( \langle expression \rangle , \langle expression \rangle )$   
                   $\mid \text{norm } ( \langle expression \rangle , \langle expression \rangle )$   
 $\langle relation \rangle ::= \langle expression \rangle \langle cmp \rangle \langle expression \rangle$   
 $\langle addop \rangle ::= + \mid -$   
 $\langle mulop \rangle ::= * \mid /$   
 $\langle cmp \rangle ::= = \mid != \mid < \mid <= \mid > \mid >=$   
 $\langle identifier \rangle ::= \langle letter \rangle \{ \langle letter \rangle \mid \langle digit \rangle \}$   
 $\langle number \rangle ::= \langle digit \rangle \{ \langle digit \rangle \}$   
 $\langle letter \rangle ::= 'a' \mid 'b' \mid \dots \mid 'z' \mid 'A' \mid 'B' \mid \dots \mid 'Z'$   
 $\langle digit \rangle ::= '0' \mid '1' \mid '2' \mid '3' \mid '4' \mid '5' \mid '6' \mid '7' \mid '8' \mid '9'$ 
```

Ключевое слово **seed** является оператором, поскольку, аналогично **write**, оно производит побочный эффект и не возвращает значение на стек. Функции **rand** и **norm** являются выражениями-факторами: они вычисляют значение и помещают его на вершину стека, что позволяет использовать в составе произвольных арифметических выражений.

1.3.3 Синтаксическая диаграмма

На рисунках 1-2 представлена синтаксическая диаграмма.

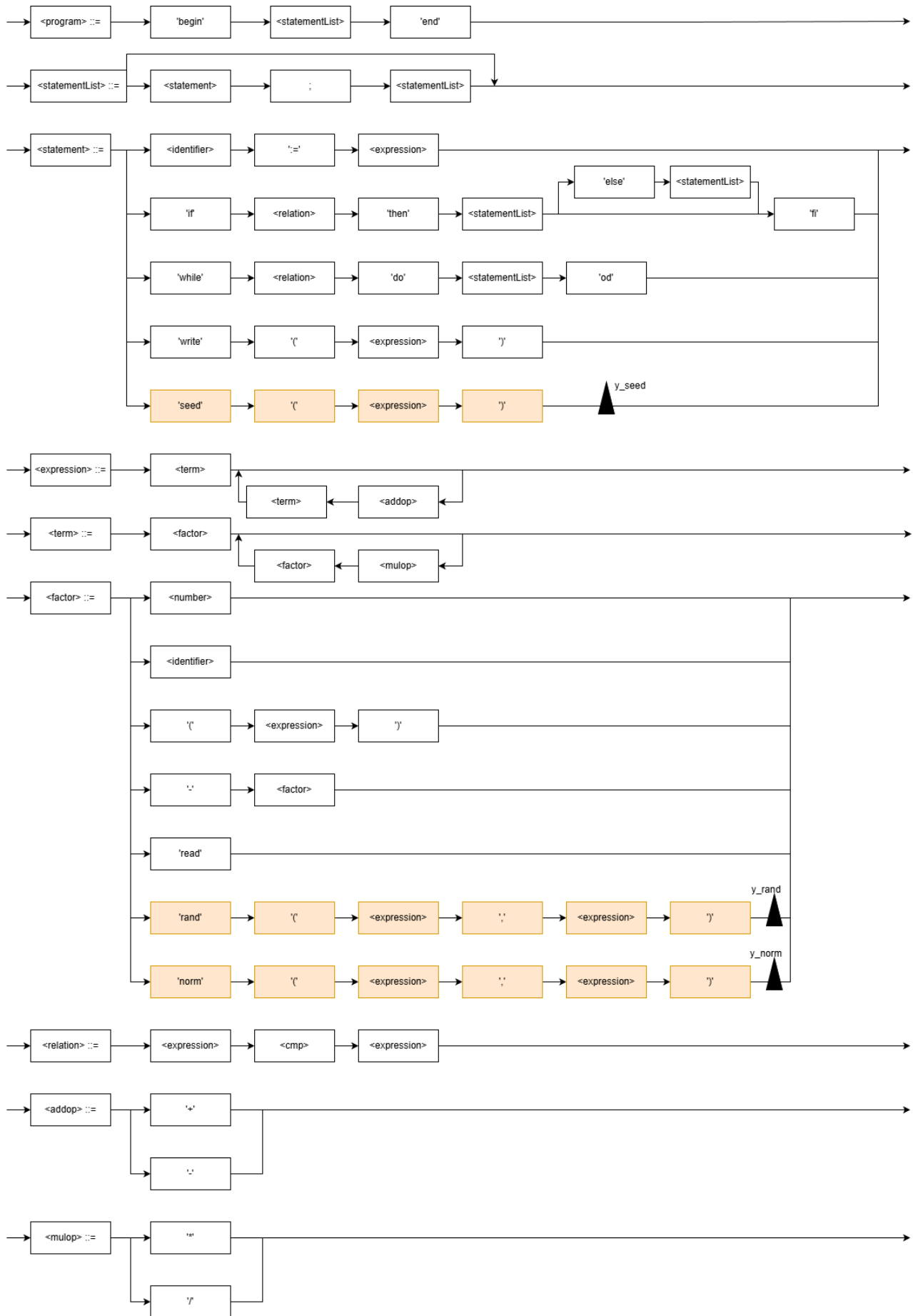


Рис. 1. Синтаксическая диаграмма языка Milan

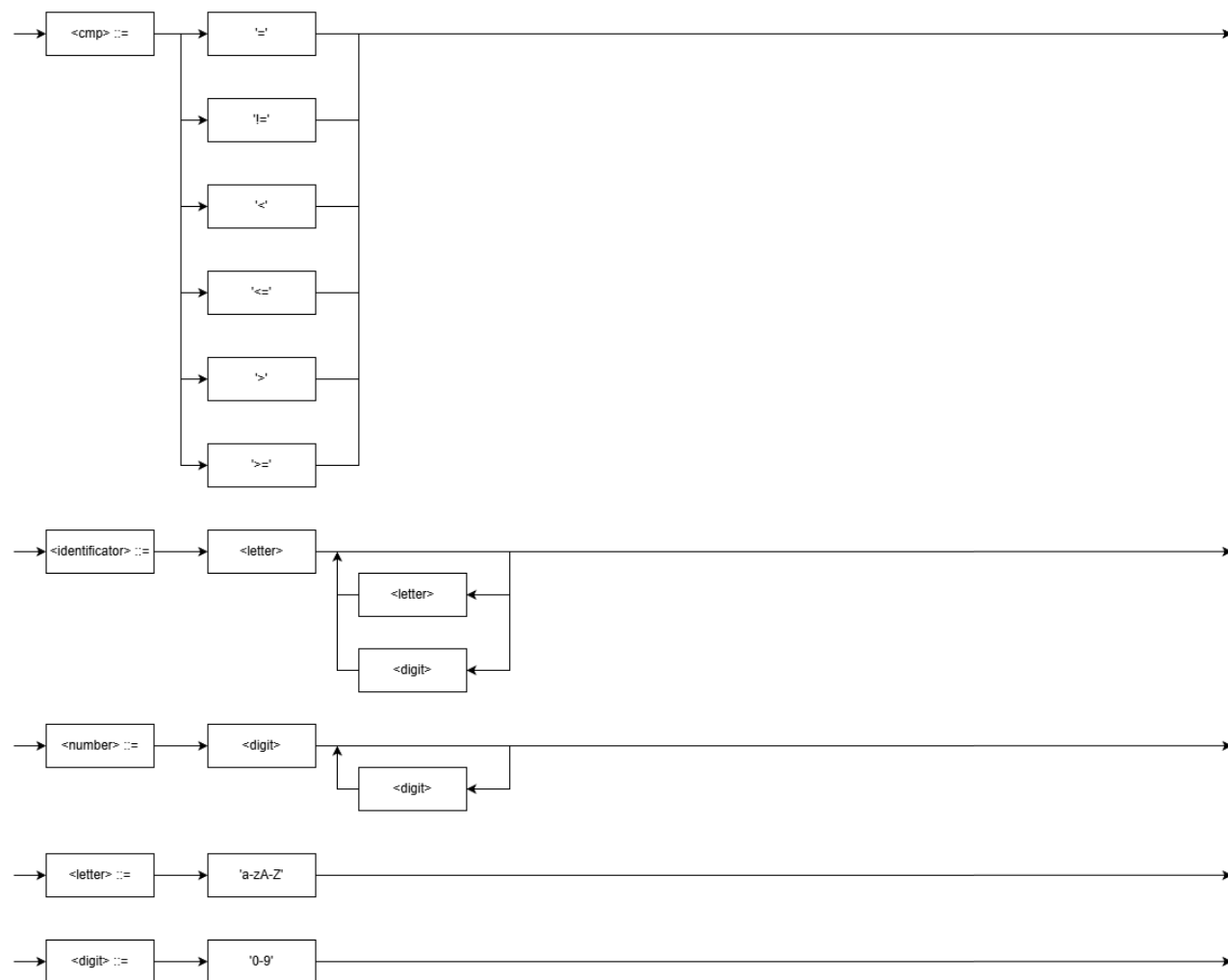


Рис. 2. Синтаксическая диаграмма языка Milan

- `y_seed`: вычисленный операнд + команда **SEED**.
- `y_rand`: два вычисленных операнда + команда **RAND**.
- `y_norm`: два вычисленных операнда + команда **NORM**.

2 Особенности реализации

2.1 Определение новых токенов

Для поддержки ключевых слов `rand`, `norm` и `seed` в лексический анализатор были добавлены три новых токена — `T_RAND`, `T_NORM` и `T_SEED` — а также токен разделителя аргументов `T_COMMA`.

Полный код перечисления `Token` с добавленными значениями приведён в листинге 1.

Листинг 1. Перечисление `Token` (`scanner.h`)

```
1 enum Token {
2     T_EOF,
3     T_ILLEGAL,
4     T_IDENTIFIER,
5     T_NUMBER,
6     T_BEGIN,
7     T_END,
8     T_IF,
9     T_THEN,
10    T_ELSE,
11    T_FI,
12    T_WHILE,
13    T_DO,
14    T_OD,
15    T_WRITE,
16    T_READ,
17    T_ASSIGN,
18    T_ADDOP,
19    T_MULOP,
20    T_CMP,
21    T_LPAREN,
22    T_RPAREN,
23    T_SEMICOLON,
24    T_RAND,
25    T_NORM,
26    T_SEED,
27    T_COMMA
28 };
```

Для корректного формирования сообщений об ошибках в массив `tokenNames_` добавлены строки, соответствующие новым токенам (листинг 2).

Листинг 2. Массив `tokenNames_` (`scanner.cpp`)

```
1 static const char * tokenNames_[] = {
2     "end of file",
3     "illegal token",
4     "identifier",
5     "number",
6     "'BEGIN'",
7     "'END'",
8     "'IF'",
9     "'THEN'",
10    "'ELSE'",
11    "'FI'",
12    "'WHILE'",
```

```

13     " 'DO' ",
14     " 'OD' ",
15     " 'WRITE' ",
16     " 'READ' ",
17     " ':= ' ",
18     " '+ ' or ' -' ",
19     " '* ' or ' /' ",
20     "comparison operator",
21     " '(',
22     " ')'",
23     " ';' ",
24     " 'RAND' ",
25     " 'NORM' ",
26     " 'SEED' ",
27     " ', ' "
28 };

```

Регистрация новых ключевых слов в словаре `keywords_` конструктора класса `Scanner` показана в листинге 3.

Листинг 3. Конструктор класса `Scanner` (`scanner.h`)

```

1 explicit Scanner(const string& fileName, istream& input)
2     : fileName_(fileName), lineNumber_(1), input_(input)
3 {
4     keywords_["begin"] = T_BEGIN;
5     keywords_["end"]   = T_END;
6     keywords_["if"]    = T_IF;
7     keywords_["then"]  = T_THEN;
8     keywords_["else"]  = T_ELSE;
9     keywords_["fi"]    = T_FI;
10    keywords_["while"]  = T_WHILE;
11    keywords_["do"]     = T_DO;
12    keywords_["od"]     = T_OD;
13    keywords_["write"]  = T_WRITE;
14    keywords_["read"]   = T_READ;
15    keywords_["rand"]   = T_RAND;
16    keywords_["norm"]   = T_NORM;
17    keywords_["seed"]   = T_SEED;
18
19    nextChar();
20 }

```

В метод `nextToken()` добавлена ветка для распознавания запятой. Полный фрагмент блока `switch` с добавленной обработкой показан в листинге 4.

Листинг 4. Фрагмент `nextToken()`: обработка символов-разделителей (`scanner.cpp`)

```

1 case '(':
2     token_ = T_LPAREN;
3     nextChar();
4     break;
5 case ')':
6     token_ = T_RPAREN;
7     nextChar();
8     break;
9 case ';':
10    token_ = T_SEMICOLON;
11    nextChar();
12    break;
13 case ',':

```

```

14 token_ = T_COMMA;
15 nextChar();
16 break;

```

2.2 Добавление инструкций в генератор кода

В перечисление `Instruction` файла `codegen.h` добавлены три новые константы (листинг 5).

Листинг 5. Перечисление `Instruction` (`codegen.h`)

```

1 enum Instruction
2 {
3     NOP,
4     STOP,
5     LOAD,
6     STORE,
7     BLOAD,
8     BSTORE,
9     PUSH,
10    POP,
11    DUP,
12    ADD,
13    SUB,
14    MULT,
15    DIV,
16    INVERT,
17    COMPARE,
18    JUMP,
19    JUMP_YES,
20    JUMP_NO,
21    INPUT,
22    PRINT,
23    RAND,
24    NORM,
25    SEED
26 };

```

В методе `Command::print()` файла `codegen.cpp` добавлены ветви `case` для вывода новых инструкций при дизассемблировании программы. Полная реализация метода приведена в листинге 6.

Листинг 6. Метод `Command::print()` (`codegen.cpp`)

```

1 void Command::print(int address, ostream& os)
2 {
3     os << address << ":\t";
4     switch(instruction_) {
5         case NOP:      os << "NOP";          break;
6         case STOP:     os << "STOP";         break;
7         case LOAD:      os << "LOAD\t" << arg_; break;
8         case STORE:     os << "STORE\t" << arg_; break;
9         case BLOAD:     os << "BLOAD\t" << arg_; break;
10        case BSTORE:    os << "BSTORE\t" << arg_; break;
11        case PUSH:      os << "PUSH\t" << arg_; break;
12        case POP:       os << "POP";          break;
13        case DUP:       os << "DUP";          break;
14        case ADD:       os << "ADD";          break;

```

```

15     case SUB:      os << "SUB";          break;
16     case MULT:    os << "MULT";         break;
17     case DIV:     os << "DIV";          break;
18     case INVERT:  os << "INVERT";        break;
19     case COMPARE: os << "COMPARE\t" << arg_; break;
20     case JUMP:    os << "JUMP\t" << arg_; break;
21     case JUMP_YES: os << "JUMP_YES\t" << arg_; break;
22     case JUMP_NO: os << "JUMP_NO\t" << arg_; break;
23     case INPUT:   os << "INPUT";         break;
24     case PRINT:   os << "PRINT";        break;
25     case RAND:    os << "RAND";         break;
26     case NORM:    os << "NORM";         break;
27     case SEED:    os << "SEED";         break;
28 }
29 os << endl;
30 }

```

2.3 Расширение синтаксического анализатора

Функции rand и norm в методе factor()

Оба вызова являются выражениями-факторами: они вычисляют значение и помещают его на стек, поэтому обрабатываются в методе `factor()`. Каждый разбирает два аргумента через запятую и генерирует соответствующую инструкцию виртуальной машины. Полная реализация метода с добавленными ветвями приведена в листинге 7.

Листинг 7. Метод `factor()` (`parser.cpp`)

```

1 void Parser::factor()
2 {
3     if(see(T_NUMBER)) {
4         int value = scanner_->getIntValue();
5         next();
6         codegen_->emit(PUSH, value);
7     }
8     else if(see(T_IDENTIFIER)) {
9         int varAddress = findOrAddVariable(
10             scanner_->getStringValue());
11         next();
12         codegen_->emit(LOAD, varAddress);
13     }
14     else if(see(T_ADDOP) &&
15         scanner_->getArithmeticValue() == A_MINUS) {
16         next();
17         factor();
18         codegen_->emit(INVERT);
19     }
20     else if(match(T_LPAREN)) {
21         expression();
22         mustBe(T_RPAREN);
23     }
24     else if(match(T_READ)) {
25         codegen_->emit(INPUT);
26     }
27     else if(match(T_RAND)) {
28         mustBe(T_LPAREN);
29         expression();

```

```

30     mustBe(T_COMMA);
31     expression();
32     mustBe(T_RPAREN);
33     codegen_ -> emit(RAND);
34 }
35 else if(match(T_NORM)) {
36     mustBe(T_LPAREN);
37     expression();
38     mustBe(T_COMMA);
39     expression();
40     mustBe(T_RPAREN);
41     codegen_ -> emit(NORM);
42 }
43 else {
44     reportError("expression expected.");
45 }
46 }

```

Оператор seed в методе statement()

Установка зерна является оператором, а не выражением — она не возвращает значение на стек, но производит побочный эффект (изменение состояния генератора). По аналогии с оператором `write` обработка выполняется в методе `statement()`. Полная реализация метода с добавленной ветвью приведена в листинге 8.

Листинг 8. Метод `statement()` (`parser.cpp`)

```

1 void Parser::statement()
2 {
3     if(see(T_IDENTIFIER)) {
4         int varAddress = findOrAddVariable(
5             scanner_ -> getStringValue());
6         next();
7         mustBe(T_ASSIGN);
8         expression();
9         codegen_ -> emit(STORE, varAddress);
10    }
11    else if(match(T_IF)) {
12        relation();
13        int jumpNoAddress = codegen_ -> reserve();
14        mustBe(T_THEN);
15        statementList();
16        if(match(T_ELSE)) {
17            int jumpAddress = codegen_ -> reserve();
18            codegen_ -> emitAt(jumpNoAddress, JUMP_NO,
19                codegen_ -> getCurrentAddress());
20            statementList();
21            codegen_ -> emitAt(jumpAddress, JUMP,
22                codegen_ -> getCurrentAddress());
23        }
24        else {
25            codegen_ -> emitAt(jumpNoAddress, JUMP_NO,
26                codegen_ -> getCurrentAddress());
27        }
28        mustBe(T_FI);
29    }
30    else if(match(T_WHILE)) {
31        int conditionAddress = codegen_ -> getCurrentAddress();
32        relation();

```

```

33     int jumpNoAddress = codegen_ -> reserve();
34     mustBe(T_D0);
35     statementList();
36     mustBe(T_OD);
37     codegen_ -> emit(JUMP, conditionAddress);
38     codegen_ -> emitAt(jumpNoAddress, JUMP_NO,
39                       codegen_ -> getCurrentAddress());
40 }
41 else if(match(T_WRITE)) {
42     mustBe(T_LPAREN);
43     expression();
44     mustBe(T_RPAREN);
45     codegen_ -> emit(PRINT);
46 }
47 else if(match(T_SEED)) {
48     mustBe(T_LPAREN);
49     expression();
50     mustBe(T_RPAREN);
51     codegen_ -> emit(SEED);
52 }
53 else {
54     reportError("statement expected.");
55 }
56 }

```

2.4 Реализация инструкций в виртуальной машине

Перечисление operation

В перечисление operation файла vm.h добавлены три опкода. Полный код перечисления приведён в листинге 9.

Листинг 9. Перечисление operation (vm.h)

```

1  typedef enum {
2      NOP = 0,
3      STOP,
4      LOAD,
5      STORE,
6      BLOAD,
7      BSTORE,
8      PUSH,
9      POP,
10     DUP,
11     INVERT,
12     ADD,
13     SUB,
14     MULT,
15     DIV,
16     COMPARE,
17     JUMP,
18     JUMP_YES,
19     JUMP_NO,
20     INPUT,
21     PRINT,
22     RAND,
23     NORM,
24     SEED
25 } operation;

```

Таблица опкодов

В таблицу `opcodes_table` добавлены записи для трёх новых инструкций. Полная таблица приведена в листинге 10.

Листинг 10. Таблица `opcodes_table` (`vm.c`)

```
1 opcode_info opcodes_table[] = {
2     {"NOP",      0},
3     {"STOP",     0},
4     {"LOAD",     1},
5     {"STORE",    1},
6     {"BLOAD",    1},
7     {"BSTORE",   1},
8     {"PUSH",     1},
9     {"POP",      0},
10    {"DUP",       0},
11    {"INVERT",    0},
12    {"ADD",       0},
13    {"SUB",       0},
14    {"MULT",      0},
15    {"DIV",       0},
16    {"COMPARE",   1},
17    {"JUMP",      1},
18    {"JUMP_YES",  1},
19    {"JUMP_NO",   1},
20    {"INPUT",     0},
21    {"PRINT",     0},
22    {"RAND",      0},
23    {"NORM",      0},
24    {"SEED",      0}
25 };
```

Исполнение инструкций в `vm_run_command()`

Инструкция `RAND` снимает два значения со стека (b — вершина, a — под ней), нормализует порядок и кладёт равномерно распределённое целое из $[\min(a, b), \max(a, b)]$. Инструкция `NORM` снимает σ и μ , применяет метод Бокса–Мюллера и возвращает результат, приведённый к целому. Инструкция `SEED` снимает одно значение; при значении 0 зерно берётся от системного времени. Полная реализация трёх инструкций в функции `vm_run_command()` приведена в листинге 11.

Листинг 11. Реализация `RAND`, `NORM`, `SEED` в `vm_run_command()` (`vm.c`)

```
1 int vm_run_command()
2 {
3     unsigned int index = vm_command_pointer;
4     operation    op     = vm_program[index].operation;
5     unsigned int arg    = vm_program[index].arg;
6     int data;
7
8     switch(op) {
9     case NOP:
10         break;
11     case STOP:
12         return 0;
13     case LOAD:
14         vm_push(vm_load(arg));
15         break;
```



```

16  case STORE:
17      vm_store(arg, vm_pop());
18      break;
19  case BLOAD:
20      vm_push(vm_load(arg + vm_pop()));
21      break;
22  case BSTORE:
23      data = vm_pop();
24      vm_store(arg + data, vm_pop());
25      break;
26  case PUSH:
27      vm_push(arg);
28      break;
29  case POP:
30      vm_pop();
31      break;
32  case DUP:
33      data = vm_pop();
34      vm_push(data); vm_push(data);
35      break;
36  case INVERT:
37      vm_push(-vm_pop());
38      break;
39  case ADD:
40      data = vm_pop(); vm_push(vm_pop() + data);
41      break;
42  case SUB:
43      data = vm_pop(); vm_push(vm_pop() - data);
44      break;
45  case MULT:
46      data = vm_pop(); vm_push(vm_pop() * data);
47      break;
48  case DIV:
49      data = vm_pop();
50      if(0 == data) vm_error(DIVISION_BY_ZERO);
51      else vm_push(vm_pop() / data);
52      break;
53  case COMPARE:
54      data = vm_pop();
55      switch(arg) {
56          case EQ: vm_push((vm_pop() == data) ? 1 : 0); break;
57          case NE: vm_push((vm_pop() != data) ? 1 : 0); break;
58          case LT: vm_push((vm_pop() < data) ? 1 : 0); break;
59          case GT: vm_push((vm_pop() > data) ? 1 : 0); break;
60          case LE: vm_push((vm_pop() <= data) ? 1 : 0); break;
61          case GE: vm_push((vm_pop() >= data) ? 1 : 0); break;
62          default: vm_error(BAD_RELATION);
63      }
64      break;
65  case JUMP:
66      if(arg < MAX_PROGRAM_SIZE)
67          { vm_command_pointer = arg; return 1; }
68      else vm_error(BAD_CODE_ADDRESS);
69      break;
70  case JUMP_YES:
71      if(arg < MAX_PROGRAM_SIZE)
72          { data = vm_pop(); if(data)
73              { vm_command_pointer = arg; return 1; } }
74      else vm_error(BAD_CODE_ADDRESS);
75      break;

```

```

76     case JUMP_NO:
77         if(arg < MAX_PROGRAM_SIZE)
78             { data = vm_pop(); if(!data)
79                 { vm_command_pointer = arg; return 1; } }
80         else vm_error(BAD_CODE_ADDRESS);
81         break;
82     case INPUT:
83         vm_push(vm_read());
84         break;
85     case PRINT:
86         vm_write(vm_pop());
87         break;
88     case RAND:
89     {
90         int b = vm_pop();
91         int a = vm_pop();
92         if (a > b) { int tmp = a; a = b; b = tmp; }
93         vm_push(a + rand() % (b - a + 1));
94         break;
95     }
96     case NORM:
97     {
98         int sigma = vm_pop();
99         int mu = vm_pop();
100         double u1 = (double)rand() / RAND_MAX;
101         double u2 = (double)rand() / RAND_MAX;
102         double z0 = sqrt(-2.0 * log(u1))
103                 * cos(2.0 * 3.1415926535 * u2);
104         vm_push((int)(mu + z0 * sigma));
105         break;
106     }
107     case SEED:
108     {
109         int seed_val = vm_pop();
110         if (seed_val < 0) {
111             vm_error(BAD_SEED);
112         }
113         if (seed_val == 0)
114             seed_val = (int)time(NULL);
115         srand(seed_val);
116         break;
117     }
118     default:
119         vm_error(UNKNOWN_COMMAND);
120 }
121
122 ++vm_command_pointer;
123 return 1;
124 }

```

Контроль допустимости аргумента оператора seed

Поскольку функция `srand()` принимает аргумент типа `unsigned int`, передача отрицательного значения приводит к неявному преобразованию типа и инициализации генератора непредсказуемым зерном. Для предотвращения подобного поведения в виртуальную машину добавлена проверка значения аргумента оператора `seed` во время выполнения.

В перечисление `runtime_error` добавлен новый код ошибки (листинг 12).

Листинг 12. Добавление кода ошибки BAD_SEED (vm.c)

```
1 typedef enum {
2     BAD_DATA_ADDRESS ,
3     BAD_CODE_ADDRESS ,
4     BAD_RELATION ,
5     STACK_OVERFLOW ,
6     STACK_EMPTY ,
7     DIVISION_BY_ZERO ,
8     BAD_INPUT ,
9     UNKNOWN_COMMAND ,
10    BAD_SEED
11 } runtime_error;
```

В функцию `vm_error()` добавлена ветвь вывода сообщения об ошибке (листинг 13).

Листинг 13. Сообщение об ошибке BAD_SEED в `vm_error()` (vm.c)

```
1 void vm_error(runtime_error error)
2 {
3     opcode_info* info;
4
5     switch(error) {
6     case BAD_DATA_ADDRESS:
7         fprintf(stderr, "Error: illegal data address\n");
8         break;
9
10    case BAD_CODE_ADDRESS:
11        fprintf(stderr, "Error: illegal address in JUMP* instruction\n");
12        break;
13
14    case BAD_RELATION:
15        fprintf(stderr, "Error: illegal comparison operator\n");
16        break;
17
18    case STACK_OVERFLOW:
19        fprintf(stderr, "Error: stack overflow\n");
20        break;
21
22    case STACK_EMPTY:
23        fprintf(stderr, "Error: stack is empty (no arguments are\n");
24        fprintf(stderr, "available)\n");
25        break;
26
27    case DIVISION_BY_ZERO:
28        fprintf(stderr, "Error: division by zero\n");
29        break;
30
31    case BAD_INPUT:
32        fprintf(stderr, "Error: illegal input\n");
33        break;
34
35    case UNKNOWN_COMMAND:
36        fprintf(stderr, "Error: unknown command, unable to execute\n");
37        break;
38
39    case BAD_SEED:
40        fprintf(stderr, "Error: seed value must be non-negative\n");
```

```

40         break;
41
42     default:
43         fprintf(stderr, "Error: runtime error %d\n", error);
44     }
45
46     fprintf(stderr, "Code:\n\n");
47
48     info = operation_info(vm_program[vm_command_pointer].operation);
49     if(NULL == info) {
50         fprintf(stderr, "%d\t(%d)\t\t%d\n", vm_command_pointer,
51             vm_program[vm_command_pointer].operation,
52             vm_program[vm_command_pointer].arg);
53     }
54     else {
55         if(info->need_arg) {
56             fprintf(stderr, "\t%d\t%s\t\t%d\n",
57                 vm_command_pointer, info->name,
58                 vm_program[vm_command_pointer].arg);
59         }
60         else {
61             fprintf(stderr, "\t%d\t%s\n", vm_command_pointer,
62                 info->name);
63         }
64     }
65     milan_error("VM error");
66 }

```

2.5 Доработка файлов виртуальной машины

Виртуальная машина принимает на вход текстовое представление программы, разбираемое лексическим анализатором `vmlex.l` (Flex) и синтаксическим анализатором `vmparse.y` (Bison).

В файл `vmlex.l` добавлено правило для токена `SEED` (листинг 14).

Листинг 14. Правила лексем в `vmlex.l`

```

1  RAND      { return T_RAND; }
2  NORM      { return T_NORM; }
3  SEED      { return T_SEED; }

```

В файл `vmparse.y` добавлены объявление нового токена и грамматическое правило (листинг 15).

Листинг 15. Грамматика в `vmparse.y`

```

1  %token T_RAND
2  %token T_NORM
3  %token T_SEED
4
5  %%
6
7  line : T_INT T_COLON T_NOP
8         { put_command($1, NOP, 0); }
9        | T_INT T_COLON T_STOP
10         { put_command($1, STOP, 0); }
11        | T_INT T_COLON T_LOAD T_INT

```

```

12         { put_command($1, LOAD,      $4); }
13     | T_INT T_COLON T_STORE T_INT
14         { put_command($1, STORE,     $4); }
15     | T_INT T_COLON T_BLOAD  T_INT
16         { put_command($1, BLOAD,     $4); }
17     | T_INT T_COLON T_BSTORE T_INT
18         { put_command($1, BSTORE,    $4); }
19     | T_INT T_COLON T_PUSH   T_INT
20         { put_command($1, PUSH,      $4); }
21     | T_INT T_COLON T_POP
22         { put_command($1, POP,       0); }
23     | T_INT T_COLON T_DUP
24         { put_command($1, DUP,       0); }
25     | T_INT T_COLON T_INVERT
26         { put_command($1, INVERT,    0); }
27     | T_INT T_COLON T_ADD
28         { put_command($1, ADD,       0); }
29     | T_INT T_COLON T_SUB
30         { put_command($1, SUB,       0); }
31     | T_INT T_COLON T_MULT
32         { put_command($1, MULT,      0); }
33     | T_INT T_COLON T_DIV
34         { put_command($1, DIV,       0); }
35     | T_INT T_COLON T_COMPARE T_INT
36         { put_command($1, COMPARE,   $4); }
37     | T_INT T_COLON T_JUMP   T_INT
38         { put_command($1, JUMP,      $4); }
39     | T_INT T_COLON T_JUMP_YES T_INT
40         { put_command($1, JUMP_YES,  $4); }
41     | T_INT T_COLON T_JUMP_NO T_INT
42         { put_command($1, JUMP_NO,   $4); }
43     | T_INT T_COLON T_INPUT
44         { put_command($1, INPUT,     0); }
45     | T_INT T_COLON T_PRINT
46         { put_command($1, PRINT,     0); }
47     | T_SET T_INT T_INT
48         { set_mem($2, $3);          }
49     | T_INT T_COLON T_RAND
50         { put_command($1, RAND,      0); }
51     | T_INT T_COLON T_NORM
52         { put_command($1, NORM,      0); }
53 T_INT T_COLON T_SEED
54         { put_command(\$1, SEED,      0); }
55 ;
56 %%

```

После внесения изменений файлы после чего файлы `lex.yy.c`, `vmparse.tab.c` и `vmparse.tab.h` были регенерированы командами:

```

bison -d vmparse.y
flex  vmlex.l

```

3 Результаты работы программы

Ниже представлены результаты работы программы. В программах 1–4 приведены корректные программы, демонстрирующие сценарии использования команд `rand()`, `norm()` и `seed()`. В программах 5–7 приведены примеры обработки ошибочных программ.

Программа №1

Текст программы	Код, сгенерированный компилятором	Вывод программы
<pre>begin x := rand(1, 100); write(x) end</pre>	<pre>0: PUSH 1 1: PUSH 100 2: RAND 3: STORE 0 4: LOAD 0 5: PRINT 6: STOP</pre>	42

Программа №2

Текст программы	Код, сгенерированный компилятором	Вывод программы
<pre>begin x := norm(50, 10); write(x) end</pre>	<pre>0: PUSH 50 1: PUSH 10 2: NORM 3: STORE 0 4: LOAD 0 5: PRINT 6: STOP</pre>	16

Программа №3

Текст программы	Код, сгенерированный компилятором	Вывод программы
<pre>begin x := rand(1, 100); write(x); if x < 50 then write(0) else write(1) fi end</pre>	<pre>0: PUSH 1 1: PUSH 100 2: RAND 3: STORE 0 4: LOAD 0 5: PRINT 6: LOAD 0 7: PUSH 50 8: COMPARE 2 9: JUMP_NO 13 10: PUSH 0 11: PRINT 12: JUMP 15 13: PUSH 1 14: PRINT 15: STOP</pre>	<pre>42 0</pre>

Программа №4

Текст программы	Код, сгенерированный компилятором	Вывод программы
<pre>begin seed(42); x := rand(1, 100); write(x); y := norm(50, 10); write(y) end</pre>	<pre>0: PUSH 42 1: SEED 2: PUSH 1 3: PUSH 100 4: RAND 5: STORE 0 6: LOAD 0 7: PRINT 8: PUSH 50 9: PUSH 10 10: NORM 11: STORE 1 12: LOAD 1 13: PRINT 14: STOP</pre>	<pre>76 21</pre>

Программа №5

Текст программы	Код, сгенерированный компилятором	Вывод программы
<pre>begin x := rand(1, 6); write(x); y := rand() end</pre>	<pre>Line 4: expression expected. Line 4: ')' found while ',' expected. Line 6: expression expected. Line 6: end of file found while ')' expected. Line 6: end of file found while 'END' expected.</pre>	—

Программа №6

Текст программы	Код, сгенерированный компилятором	Вывод программы
<pre>begin x := norm(100, 15); write(x); end</pre>	<pre>Line 4: statement expected.</pre>	—

Программа №7

Текст программы	Код, сгенерированный компилятором	Вывод программы
<pre>begin seed(-1); x := rand(1, 100); write(x) end</pre>	<pre>0: PUSH 1 1: INVERT 2: SEED 3: PUSH 1 4: PUSH 100 5: RAND 6: STORE 0 7: LOAD 0 8: PRINT 9: STOP</pre>	<pre>Error^ seed value must be non-negative Code: 2 SEED VM error</pre>

Заключение

В ходе выполнения лабораторной работы компилятор языка программирования Milan, относящегося к классу контекстно-свободных, был доработан путём добавления поддержки генерации псевдослучайных чисел. Были расширены лексический и синтаксический анализаторы компилятора, а также виртуальная машина.

В лексический анализатор добавлены новые токены `T_RANDOM`, `T_NORM`, `T_SEED` и `T_COMMA`, зарегистрированы соответствующие ключевые слова, а метод `nextToken()` дополнен обработкой символа запятой. Синтаксический анализатор после доработки по-прежнему относится к типу LL(1), реализованному методом рекурсивного спуска. В метод `factor()` добавлены ветви разбора функций `rand` и `norm` как выражений-факторов, возвращающих значение на стек; в метод `statement()` добавлен оператор `seed`, управляющий состоянием генератора. В виртуальную машину добавлены три новые инструкции: `RAND`, `NORM` и `SEED`.

Достоинства:

- Функции `rand` и `norm` являются полноценными выражениями-факторами и могут использоваться в составе произвольных арифметических выражений.
- Инструкция `RAND` автоматически нормализует порядок аргументов, что предотвращает ошибку при случайной перестановке границ диапазона.
- Оператор `seed(0)` инициализирует генератор от системного времени, обеспечивая различные последовательности при каждом запуске.

Недостатки:

- Результат инструкции `NORM` приводится к целому числу, что влечёт потерю точности при малых значениях σ .
- Язык Milan оперирует исключительно целочисленной арифметикой, поэтому вещественные параметры распределений задать невозможно.

Масштабирование:

- Добавление обработки вещественных чисел в язык Milan, что позволило бы точнее задавать параметры нормального распределения и использовать результат без округления.
- Расширение набора распределений, например добавление равномерного вещественного распределения.

Список литературы

1. Секция «Телематика» – <https://tema.spbstu.ru/compiler/> (дата обращения – 30.05.2026)